

Reviewer Pack — Minimal Local Indeterminacy in Topological Quantum Field The...

Entry da7ff60219e4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 10:32:54 UTC

Minimal Local Indeterminacy in Topological Quantum Field Theory and Circuit Depth

Entry ID: da7ff60219e4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 10:32:54 UTC

1. Conjecture

Field A (mathematical branch): Topological Quantum Field Theory (TQFT) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Depth)

Statement:

For any given d -regular graph G , the minimal local indeterminacy of its associated TQFT state φ_G is linearly correlated with its circuit depth $d(\varphi_G)$, such that $\text{Indet}(\varphi_G) = \Theta(d(\varphi_G))$.

Rationale (proposer's reasoning):

Topological Quantum Field Theory (TQFT) has been applied to quantum computation, and local indeterminacy measures the complexity of the state. It is expected that this measure could provide a new perspective on circuit complexity. Circuit depth serves as a standard complexity measure in classical computing. The bridge between these two fields could reveal hidden structures in both.

Taxonomy category: TQFT_to_BooleanCircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f488788853b856ee

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given d -regular graph G , if the correlation coefficient between $\text{Indet}(\varphi_G)$ and $d(\varphi_G)$ is greater than or equal to 0.8 and the mean difference between the two metrics across 30 seeds is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            return None
        graph = [[] for _ in range(n)]
        edges_added = set()
        for i in range(n):
            for j in range(i + 1, n):
                if len(graph[i]) < d and len(graph[j]) < d:
                    if (i, j) not in edges_added and (j, i) not in edges_added:
                        graph[i].append(j)
                        graph[j].append(i)
                        edges_added.add((i, j))
        return graph

    def calculate_jones_polynomial(G):
        n = len(G)
        if n == 0:
            return 1
        if n == 1:
            return 1
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if j in G[i]:
                    M[i][j] = -1
                    M[j][i] = -1
        det = determinant(M)
        return det

    def determinant(matrix):
        n = len(matrix)
```

```

    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def calculate_local_indeterminacy(jones_polynomial):
    # Placeholder for actual computation
    return abs(jones_polynomial)

def calculate_circuit_depth(G):
    n = len(G)
    if n == 0:
        return 0
    if n == 1:
        return 1
    visited = [False] * n
    depth = 0

    def dfs(node, current_depth):
        nonlocal depth
        visited[node] = True
        for neighbor in G[node]:
            if not visited[neighbor]:
                dfs(neighbor, current_depth + 1)
        depth = max(depth, current_depth)

    for i in range(n):
        if not visited[i]:
            dfs(i, 0)

    return depth

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    G = generate_d_regular_graph(n, 3)
    if G is None:
        continue
    jones_polynomial = calculate_jones_polynomial(G)
    local_indeterminacy = calculate_local_indeterminacy(jones_polynomial)
    circuit_depth = calculate_circuit_depth(G)

    results.append({
        "n": n,
        "local_indeterminacy": local_indeterminacy,
        "circuit_depth": circuit_depth
    })

if not results:
    return {
        "metric_name": "Indet( $\phi_G$ )",
        "metric_value": None,

```

```

        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "Graph generation failed for all sizes"
    }

mean_local_indeterminacy = sum(result["local_indeterminacy"] for result in results) / len(results)
mean_circuit_depth = sum(result["circuit_depth"] for result in results) / len(results)
correlation_coefficient = 0
if mean_circuit_depth != 0:
    correlation_coefficient = (sum((result["local_indeterminacy"] - mean_local_indeterminacy)
                                * math.sqrt(sum((result["local_indeterminacy"] - mean_local_indeterminacy) ** 2
                                                + sum((result["circuit_depth"] - mean_circuit_depth) ** 2)
                                                for result in results))) / len(results))

return {
    "metric_name": "Indet( $\phi_G$ )",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation_coefficient >= 0.8 and abs(mean_local_indeterminacy - mean_circuit_depth) < 0.1,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16378
2	preregistration	ollama_remote	glm4:latest	0	0	9342
3	novelty	ollama_remote	glm4:latest	0	0	9029
4	novelty	ollama_remote	glm4:latest	0	0	8675
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46389
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15915
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15862
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16348
9	judge	ollama_remote	glm4:latest	0	0	21542

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 159480 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/da7ff60219e4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/da7ff60219e4.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/da7ff60219e4.tar.gz (if generated)